AI BUILD NOTES: WHOLESALESHOPPING.CO.UK HOMEPAGE REDESIGN

WholesaleShopping.co.uk homepage redesign · NaskSoft AI Web & CRO assessment · October 2026

TOOLS USED

Tool	What I used it for
Claude Code (Claude Opus 5.5), an AI coding agent in the terminal	Reading the brief, researching the live store and Fashion Nova, writing the plan, building the site (Astro, JavaScript, CSS), writing tests, reading Lighthouse reports, and drafting these notes from my build log
A **Claude Code helper agent**, run in the background	Collecting real, sourced facts about the store into `notes/real-facts.md`: Trustpilot score, delivery and dispatch times, VAT, minimum order, menus, FAQs
Playwright (a test tool Claude drove)	Screenshots of the current site, Fashion Nova and every build step; 55 automated tests at four screen sizes
Lighthouse	Speed and accessibility audits

MY PROCESS

1. **Understand the brief.** Claude read the PDF and turned it into checklists (`REQUIREMENTS.md`), with every requirement linked to a step in the plan.
2. **Research before designing.** Claude downloaded the live homepage, the store's public product data and Fashion Nova's homepage, then listed real problems:
   - 12 mixed menu items
   - banners with text baked into images
   - the page heading buried at the bottom
   - confusing pack prices
   - trust signals only in the footer

   And patterns worth borrowing from Fashion Nova: one offer bar, big search, category tiles, a tabbed product row, and Quick Add.
3. **Plan with checkpoints.** A phased plan (`PLAN.md`) with points where I reviewed the work, plus a `CLAUDE.md` file so every Claude session follows the plan and the rules (real facts only, mobile first).
4. **Use real data.** A script copies 70 real products (names, pack sizes, per-piece prices, photos) and the helper collected sourced facts. These facts changed the design:
   - The store blocks checkout under £100, so the bag shows that rule early.
   - The "Under £X" collections filter by pack price, so those tiles say "Packs under £X".
5. **Build in small steps.** After each step: screenshots at phone and desktop size, then fix what they showed. At checkpoint 1 I approved the look and asked for the real logo instead of a text one.

6. **Test and measure.**
    - Automated tests at four screen sizes.
    - A link checker for all 148 links.
    - Lighthouse.
    - Side-by-side before/after images.

PROMPTS AND ITERATION

A few of the prompts that shaped the project (spelling tidied):

1. *"Understand this assessment"* (with the PDF). → A summary of the task, the deliverables and the scoring. Claude pointed out that about 30% of the marks depend on these notes, so I kept a build log from the start.
2. *"Make an implementation plan first."* → The first answer was research findings and section ideas, not the step-by-step plan I wanted. I asked again: *"Implementation plan."* The second answer was a full plan file: stack, folders, specs with "done when" criteria, tests and risks.
3. *"Clone this repo ... and make an md file so the plan is added, so you can implement according to plan.md or requirements.md."* → `PLAN.md`, `REQUIREMENTS.md` and `CLAUDE.md` in the repo.
4. *"Yes, commit and push. Use the real logo."* → My feedback at checkpoint 1.
5. *"Start with the remaining phases"* and later *"No need to deploy, we'll use Netlify."* → Testing and fixing; I chose the hosting myself.

WHAT WENT WRONG

1. **A wrong assumption about the data.** Claude's product script assumed every product's size option was a pack size ("Pack of 3"). In fact 26 of the 70 products use "Regular / Plus Size". Some of those prices are for a whole pack: one product's address says "pack-of-6" and costs £24. Others are for one piece. The page would have shown wrong per-piece prices.
2. **The page was wider than the phone.** The newsletter section made the whole page lay out at 421 px on a 390 px phone, so a real phone would show it slightly zoomed out. **Claude's own test missed it**: it compared the page with `innerWidth`, which on phones grows to fit content that's too wide.
3. **The bottom bar hid the main buttons on phones.** All tests passed, because they only checked the buttons were inside the screen. I found the problem when I made the side-by-side comparison image for these notes. In the same image the offer bar read "deliveryon orders", because a layout change had removed the spaces.
4. **Slow pieces found by Lighthouse.**
    - On desktop, the slowest-loading element was the second hero photo. It was set to load lazily although it's on the first screen.
    - On phones, an 88 KB font file was competing with the hero photo for bandwidth.

5. **Problems that only appeared on the live site.**
    - The first deploy returned "401 Unauthorized": the Netlify team had login protection switched on for every deploy.
    - The tests then failed on small Android phones only. Netlify adds a "Built with Netlify" badge in the bottom corner, and on phones it covered the Saved, Bag and Account buttons.
    - Headings and paragraphs jumped when the web fonts arrived. On my computer there's no network delay, so locally this never showed.

HOW I FIXED IT

1. **Never invent a price.** Those 26 products show their price with "Regular & Plus sizes" and no per-piece figure, and the gap is listed as a follow-up.
2. **The layout and the test.** The grid columns can now shrink (`minmax(0, 1fr)`), and the test compares against the real screen width. Checked at 320, 360, 390, 768 and 1440 px.
3. **The buttons and the spaces.**
    - The hero's height now leaves room for the bottom bar, and the test checks against the bar.
    - The offer text is wrapped so the spaces stay, with a new test that checks the wording.
4. **Speed.**
    - Both desktop hero photos now load straight away.
    - The heading font is cut down to the one style the design uses (88 KB → 8 KB).
    - Photos come pre-cropped from Shopify's image server.

    Results: Lighthouse desktop performance went from 81 to 92 and phone from 90 to 91, with accessibility 100, best practices 100 and layout shift 0.
5. **The live site.**
    - Login protection now applies to preview deploys only, so the live link is public.
    - The badge is switched off.
    - The 8 KB heading font is built into the page.
    - The fallback system font is size-matched to Inter, so text doesn't move when Inter arrives. The live desktop layout shift fell from 0.072 to 0.018.
    - All 55 tests pass against the live link.

WHAT I LEARNED

- **Measure, don't just look.** Several bugs only showed up in numbers (a page 421 px wide, the slowest-loading element) or in a side-by-side image, not in a quick look at the screen.
- **A passing test can be wrong.** In my first test run, three of the four kinds of failure were mistakes in the tests. Two real bugs passed tests that measured the wrong thing.
- **CSS grid items won't shrink below their content** unless the column uses `minmax(0, 1fr)`.
- **Shopify's image server is very flexible.** It can resize and crop photos through the web address (`&width=…&height=…&crop=top`), and it automatically serves smaller WebP/AVIF files to browsers that support them.

- **In B2B, the store's real rules shape the design.** The £100 minimum and pack-price filters changed what the page says.
- **Test the live site, not just your computer.** Hosting added its own problems (login protection and a badge over my buttons), and network delays revealed a layout shift that never showed locally.
- **Be specific with AI.** "An implementation plan" got a strategy summary; asking for "a step-by-step plan with files, tasks and tests" got what I needed.

Results:

- 55 automated tests pass at four screen sizes.
- 146 of 148 links work and none are broken; the other 2 block automated checks and were checked by hand.
- Live at https://wholesaleshopping-homepage-redesign.netlify.app. Lighthouse on the live site: accessibility 100 and best practices 100. Performance is 87–96 on phones and 92–99 on desktop, varying with how fast the server responds from my location.
- About 9 KB of JavaScript.